

Module - 04

Transport Layer

4.1. Transport-Layer Services

4.2. Connectionless and Connection-Oriented Protocols

4.3. Transport-Layer Protocols

4.3.1. Introduction

4.3.2. Services

4.3.3. Port Numbers

4.4. User datagram Protocol

4.4.1. User Datagram

4.4.2. UDP Services

4.5. Transmission Control Protocol

4.5.1. TCP Services

4.5.2. TCP Features

4.5.3. Segment

4.5.4. A TCP Connection

4.5.5. Windows in TCP

4.5.8. Flow control

4.5.9. Error control

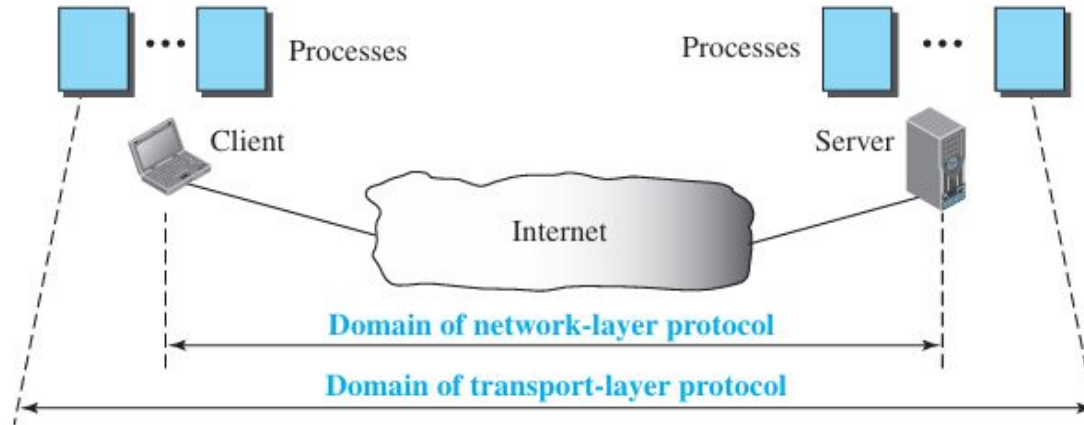
4.5.10. TCP Congestion Control

4.1. Transport-Layer Services

- The transport layer is located between the network layer and the application layer. The transport layer is responsible for providing services to the application layer; it receives services from the network layer
- In this section, we discuss the services that can be provided by the transport layer

Process-to-Process Communication

Figure 23.2 *Network layer versus transport layer*



Addressing: Port Numbers

- For example, there are clients that are assigned well-known port numbers. Every client process knows the well-known port number of the corresponding server process.
- For example, while the daytime client process, a well-known client program, can use an ephemeral (temporary) port number, 52,000, to identify itself, the daytime server process must use the well-known (permanent) port number 13. Figure 23.3 shows this concept.

Figure 23.3 *Port numbers*

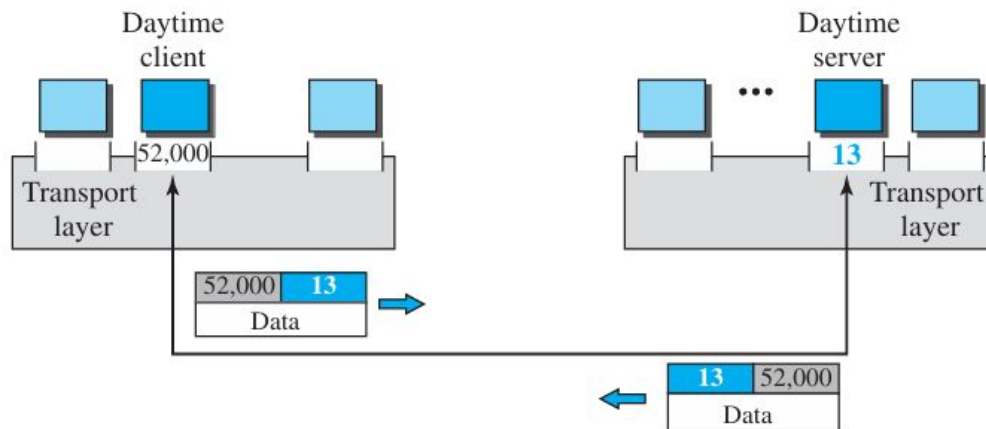
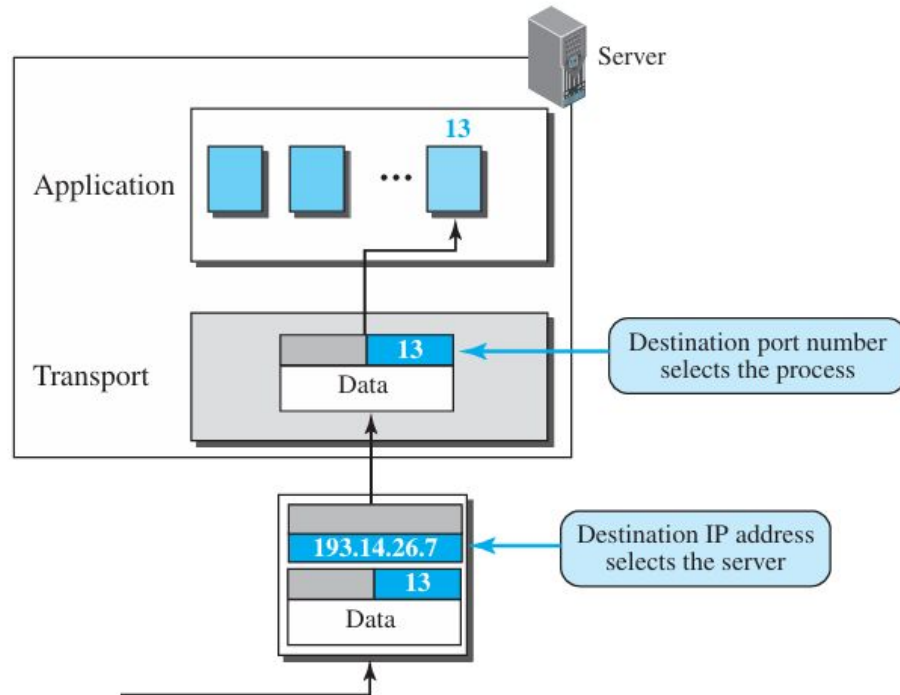
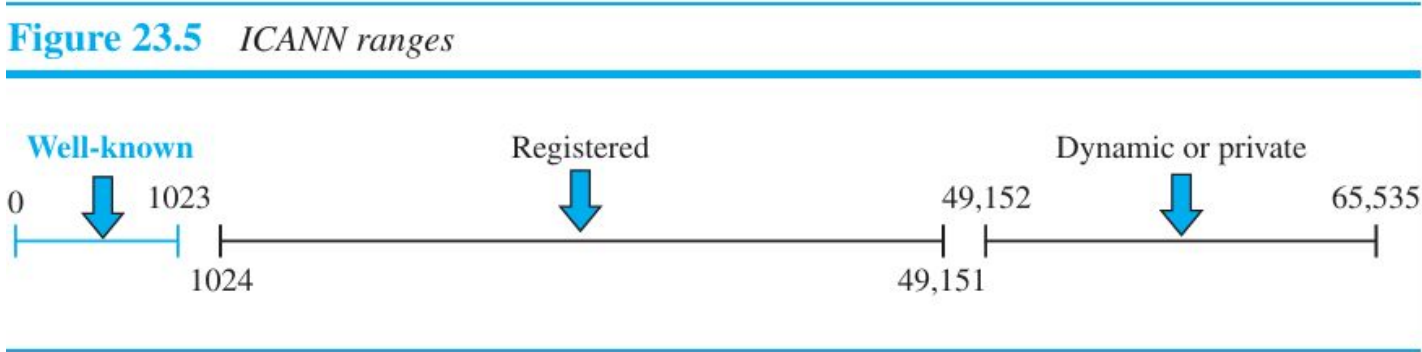


Figure 23.4 *IP addresses versus port numbers*



ICANN Ranges

- ICANN - Internet Corporation for Assigned Names and Numbers - has divided the port numbers into three ranges: well-known, registered, and dynamic (or private), as shown in Figure 23.5.



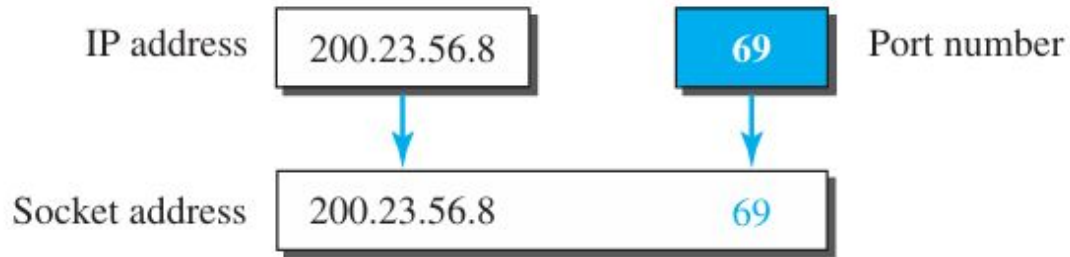
- **Well-known ports**
- **Registered ports.**
- **Dynamic ports.**

- **Example 23.1** In UNIX, the well-known ports are stored in a file called `/etc/services`. Each line in this file gives the name of the server and the well-known port number. We can use the `grep` utility to extract the line corresponding to the desired application. The following shows the port for TFTP. Note that TFTP can use port 69 on either UDP or TCP.
 - SNMP uses two port numbers (161 and 162), each for a different purpose.
 - `$grep tftp/etc/services`
 - `tftp 69/tcp`
 - `tftp 69/udp`
 - `$grep snmp/etc/services`
 - `snmp161/tcp#Simple Net Mgmt Proto`
 - `snmp161/udp#Simple Net Mgmt Proto`
 - `snmptrap162/udp#Traps for SNMP`

Socket Addresses

- The combination of an IP address and a port number is called a socket address.

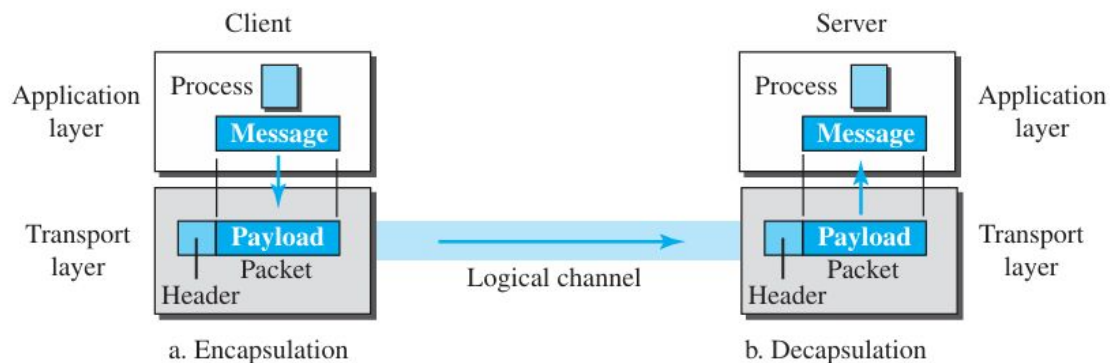
Figure 23.6 *Socket address*



Encapsulation and Decapsulation

- To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages
- **Encapsulation** happens at the sender site. When a process has a message to send, it passes the message to the transport layer along with a pair of socket addresses and some other pieces of information, which depend on the transport-layer protocol.
- The transport layer receives the data and adds the transport-layer header. The packets at the transport layer in the Internet are called user datagrams, segments, or packets, depending on what transport-layer protocol we use.
- **Decapsulation** happens at the receiver site. When the message arrives at the destination transport layer, the header is dropped and the transport layer delivers the message to the process running at the application layer.
- The sender socket address is passed to the process in case it needs to respond to the message received.

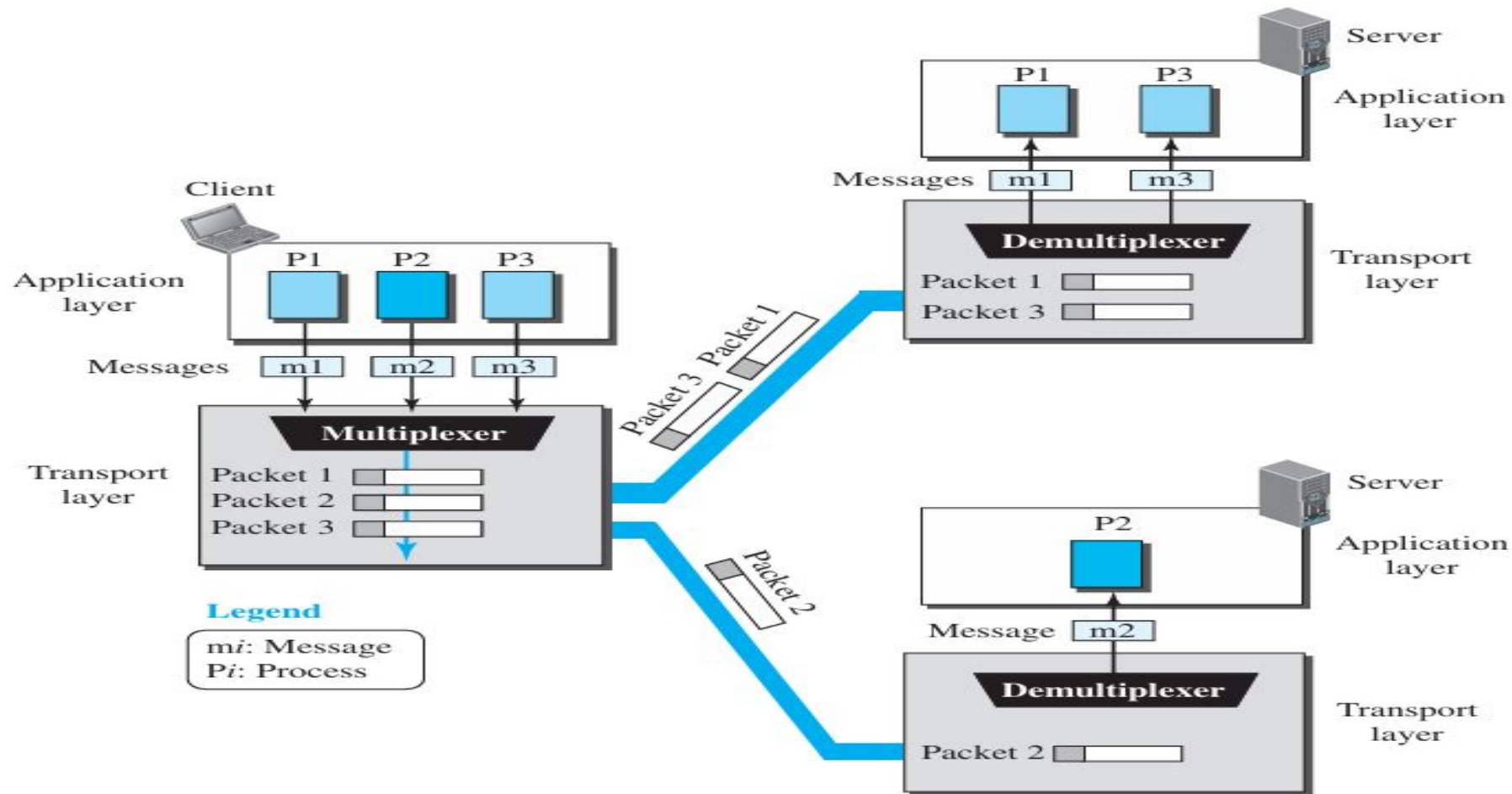
Figure 23.7 *Encapsulation and decapsulation*



Multiplexing and Demultiplexing

- Whenever an entity accepts items from more than one source, this is referred to as **multiplexing** (many to one); whenever an entity delivers items to more than one source, this is referred to as **demultiplexing** (one to many).

Figure 23.8 *Multiplexing and demultiplexing*

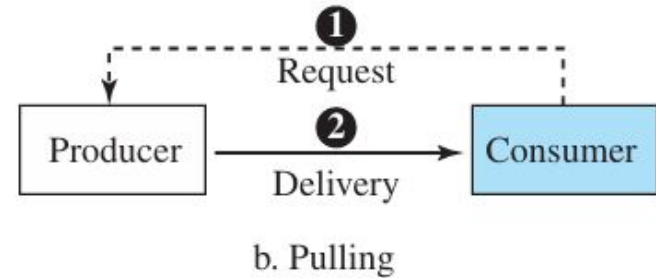
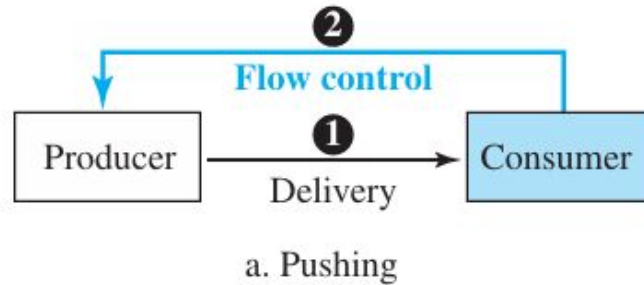


Flow Control

Pushing or Pulling

- If the sender delivers items whenever they are produced —without a prior request from the consumer —the delivery is referred to as pushing.
- If the producer delivers the items after the consumer has requested them, the delivery is referred to as pulling.

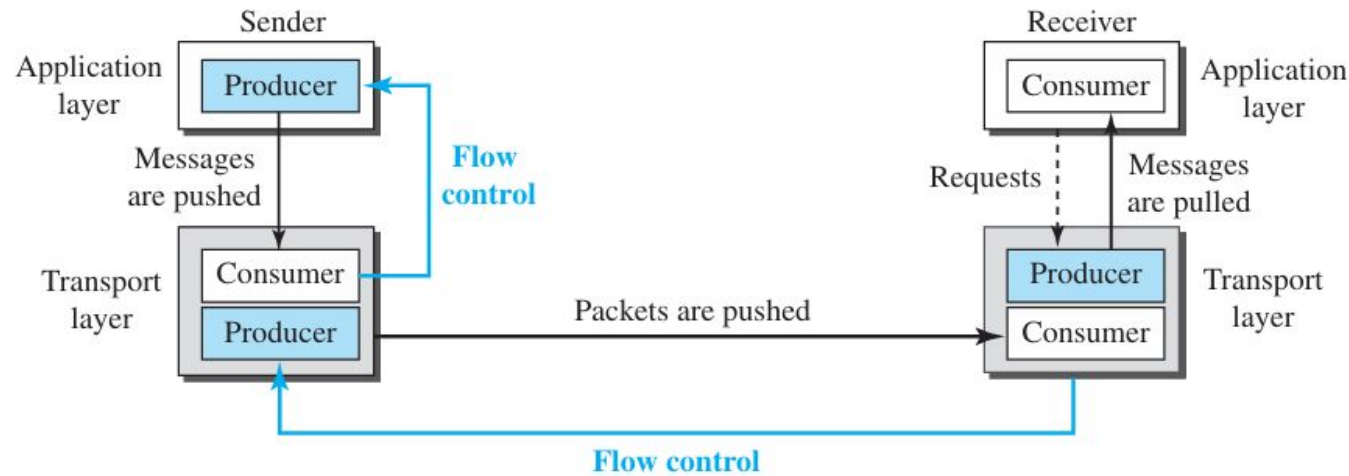
Figure 23.9 *Pushing or pulling*



Flow Control at Transport Layer

- Figure 23.10 shows that we need at least two cases of flow control: from the sending transport layer to the sending application layer and from the receiving transport layer to the sending transport layer.

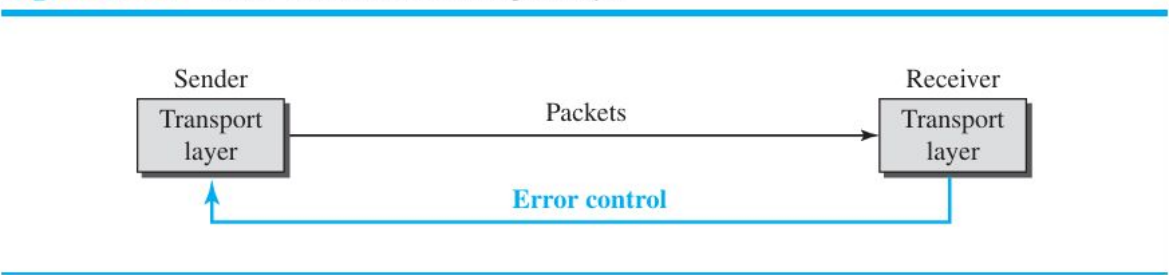
Figure 23.10 *Flow control at the transport layer*



Buffers

Error Control

Figure 23.11 *Error control at the transport layer*



Sequence Numbers

Acknowledgment

Combination of Flow and Error Control

Sliding Window

Figure 23.12 Sliding window in circular format

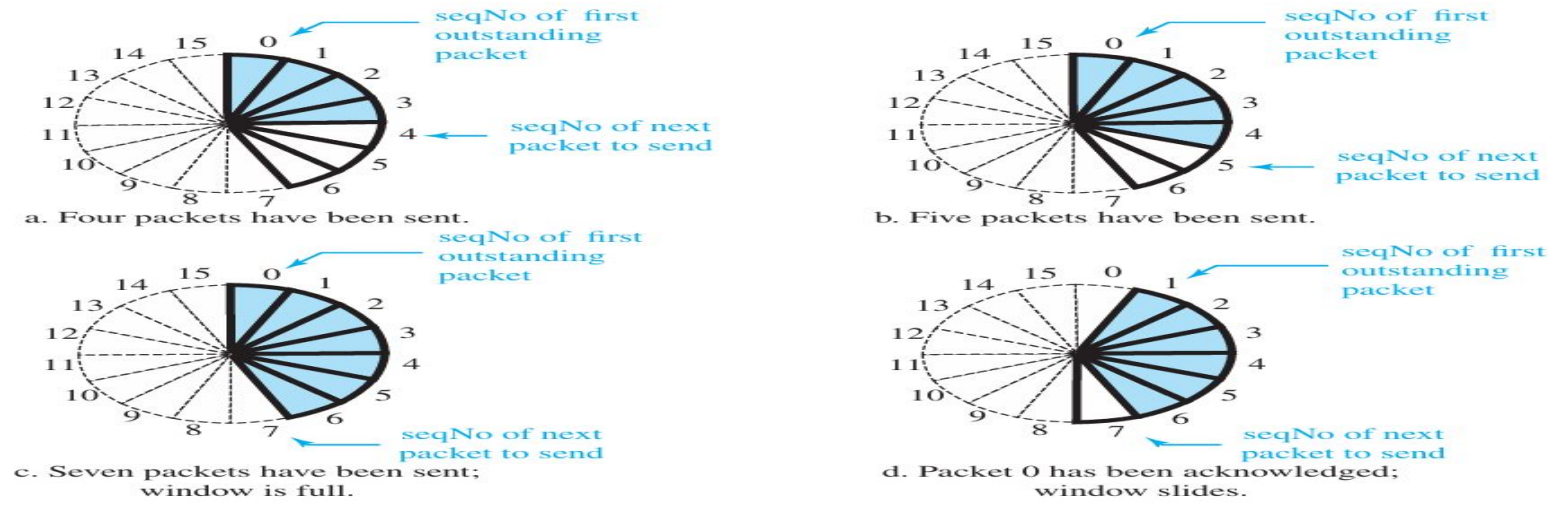
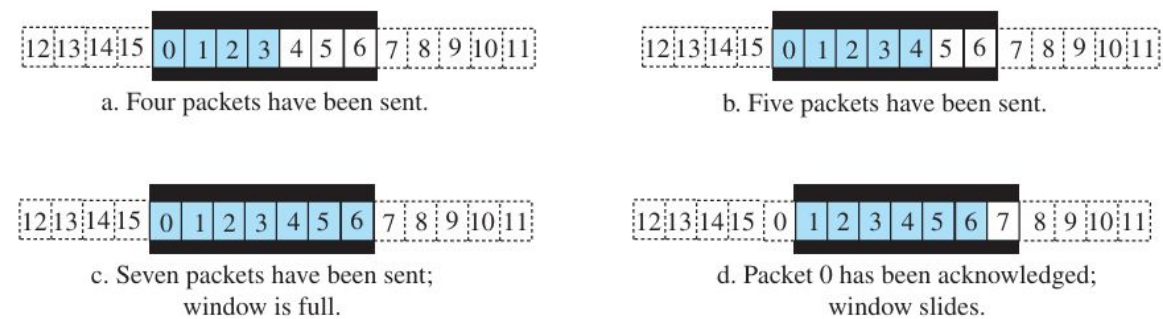


Figure 23.13 Sliding window in linear format

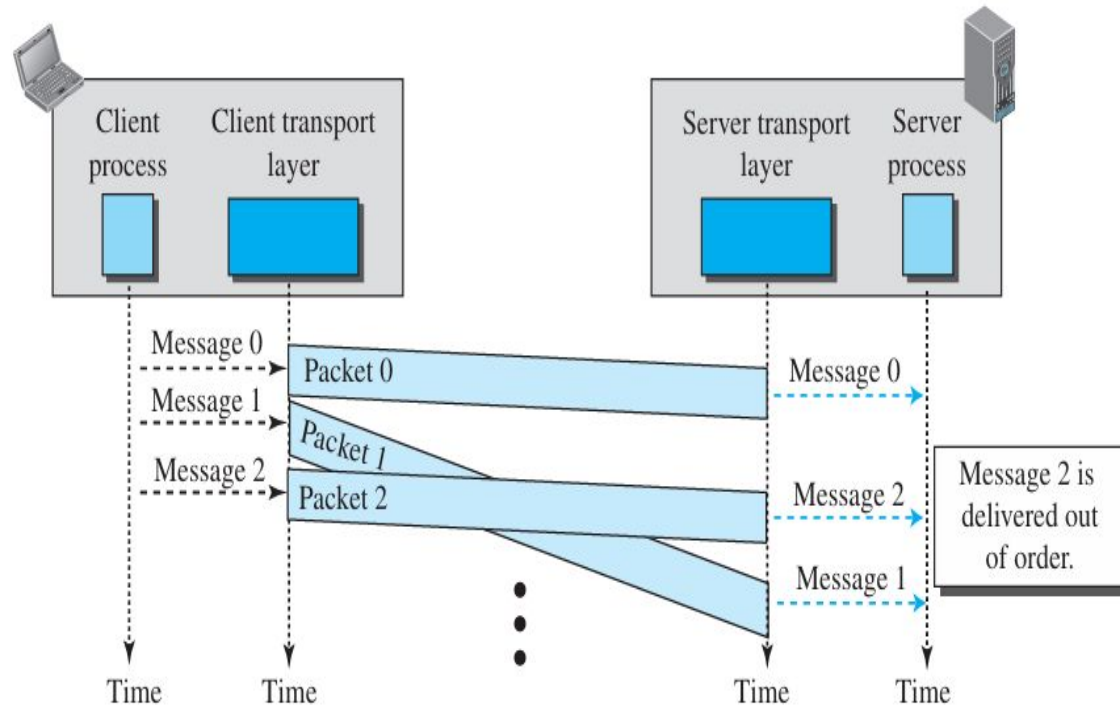


Congestion Control

4.2. Connectionless and Connection-Oriented Protocols

Connectionless Service

Figure 23.14 *Connectionless service*

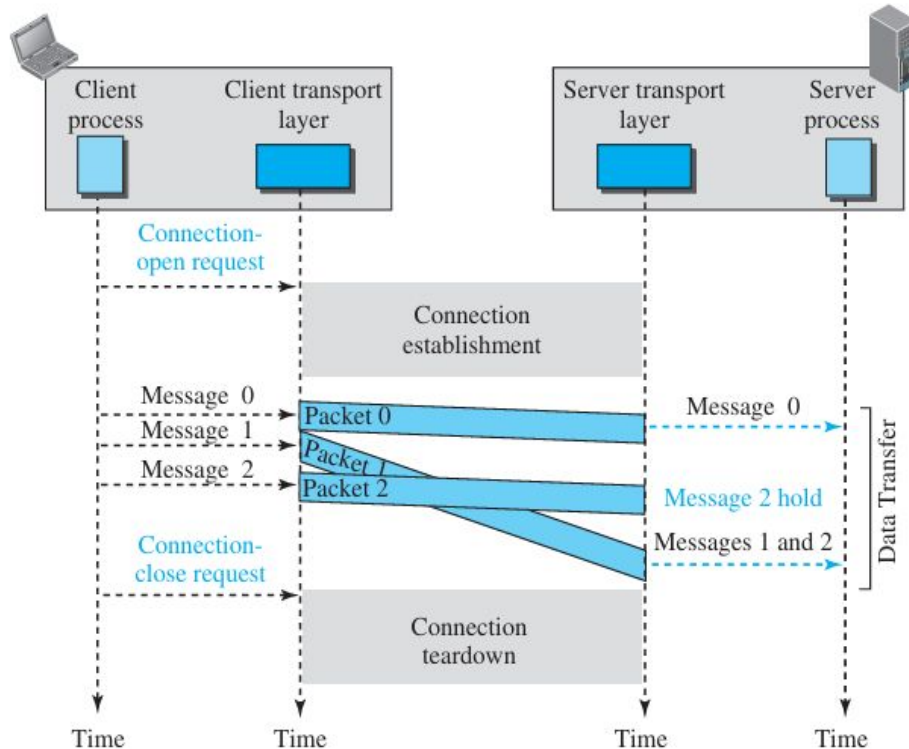


- In Figure 23.14, we have shown the movement of packets using a timeline, but we have assumed that the delivery of the process to the transport layer and vice versa are instantaneous.
- The figure shows that at the client site, the three chunks of messages are delivered to the client transport layer in order (0, 1, and 2).
- Because of the extra delay in transportation of the second packet, the delivery of messages at the server is not in order (0, 2, 1). If these three chunks of data belong to the same message, the server process may have received a strange message.

4.2. Connectionless and Connection-Oriented Protocols

Connection-Oriented Service

Figure 23.15 *Connection-oriented service*

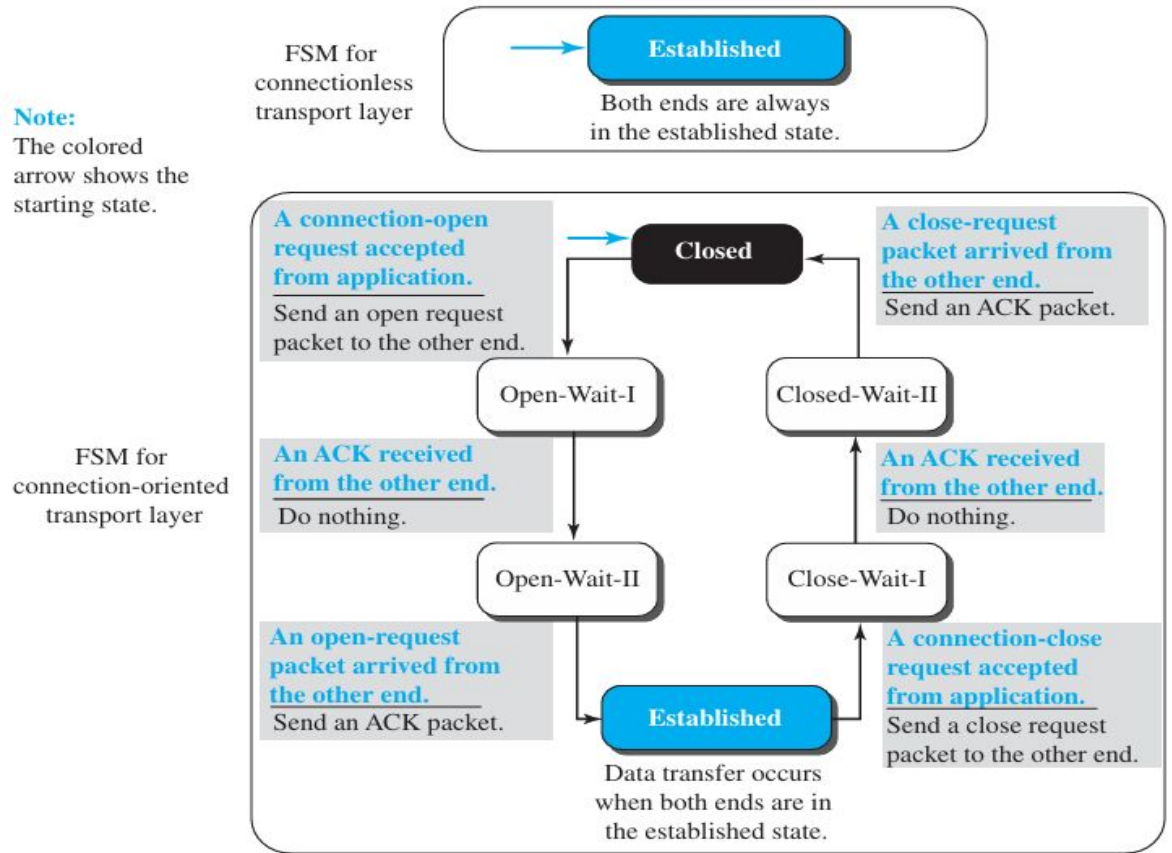


- Figure 23.15 shows the connection establishment, data-transfer, and tear-down phases in a connection-oriented service at the transport layer.
- We can implement flow control, error control, and congestion control in a connection oriented protocol.

Finite State machine

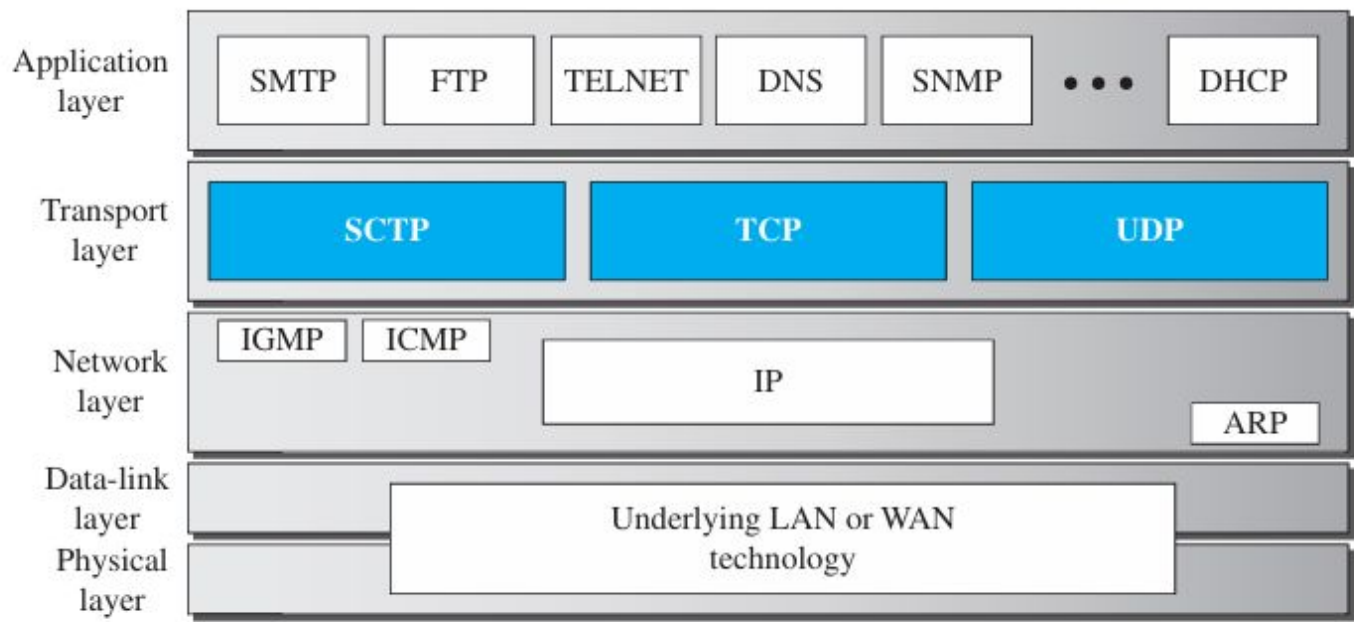
Figure 23.16 Connectionless and connection-oriented service represented as FSMs

Note:
The colored arrow shows the starting state.



4.3. Transport-Layer Protocols

Figure 24.1 *Position of transport-layer protocols in the TCP/IP protocol suite*



4.3.2. Services

- Each protocol provides a different type of service and should be used appropriately.

UDP

- UDP is an unreliable connectionless transport-layer protocol used for its simplicity and efficiency in applications where error control can be provided by the application-layer process.

TCP

- TCP is a reliable connection-oriented protocol that can be used in any application where reliability is important.

SCTP

- SCTP is a new transport-layer protocol that combines the features of UDP and TCP.

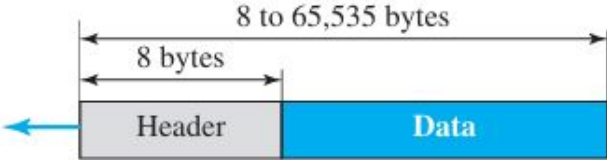
4.3.3. Port Numbers

<i>Port</i>	<i>Protocol</i>	<i>UDP</i>	<i>TCP</i>	<i>SCTP</i>	<i>Description</i>
7	Echo	√	√	√	Echoes back a received datagram
9	Discard	√	√	√	Discards any datagram that is received
11	Users	√	√	√	Active users
13	Daytime	√	√	√	Returns the date and the time
17	Quote	√	√	√	Returns a quote of the day
19	Chargen	√	√	√	Returns a string of characters
20	FTP-data		√	√	File Transfer Protocol
21	FTP-21		√	√	File Transfer Protocol
23	TELNET		√	√	Terminal Network
25	SMTP		√	√	Simple Mail Transfer Protocol
53	DNS	√	√	√	Domain Name Service
67	DHCP	√	√	√	Dynamic Host Configuration Protocol
69	TFTP	√	√	√	Trivial File Transfer Protocol
80	HTTP		√	√	HyperText Transfer Protocol
111	RPC	√	√	√	Remote Procedure Call
123	NTP	√	√	√	Network Time Protocol
161	SNMP-server	√			Simple Network Management Protocol
162	SNMP-client	√			Simple Network Management Protocol

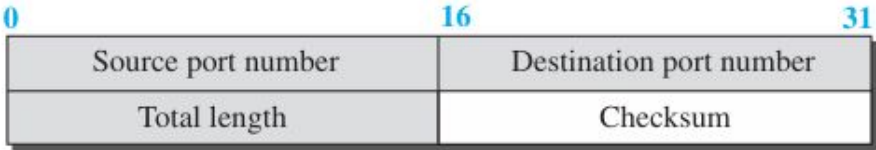
4.4. User datagram Protocol

4.4.1. User Datagram

Figure 24.2 *User datagram packet format*



a. UDP user datagram



b. Header format

- **Example 24.1** The following is the content of a UDP header in hexadecimal format.

CB84000D001C001C

- **Solution :** a. The source port number is the first four hexadecimal digits $(CB84)_{16}$, which means that the source port number is 52100.
- b. The destination port number is the second four hexadecimal digits $(000D)_{16}$, which means that the destination port number is 13.
- c. The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- d. The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
- e. Since the destination port number is 13 (well-known port), the packet is from the client to the server.
- f. The client process is the Daytime (see Table 24.1).

4.4.2. UDP Services

Process-to-Process Communication

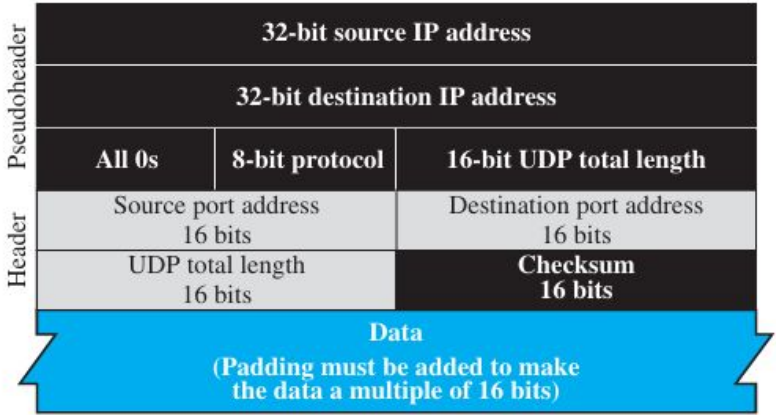
Connectionless Services

Flow Control

Error Control

Checksum

Figure 24.3 *Pseudoheader for checksum calculation*



4.4.2. UDP Services

Example 24.2

- What value is sent for the checksum in each one of the following hypothetical situations?
 - a. The sender decides not to include the checksum.
 - b. The sender decides to include the checksum, but the value of the sum is all 1s.
 - c. The sender decides to include the checksum, but the value of the sum is all 0s.

Solution

- a. The value sent for the checksum field is all 0s to show that the checksum is not calculated.
- b. When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s. The second complement operation is needed to avoid confusion with the case in part a.
- c. This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudo header have nonzero values.

4.4.2. UDP Services

Congestion Control

Encapsulation and Decapsulation

Queuing

Multiplexing and Demultiplexing

4.5. Transmission Control Protocol

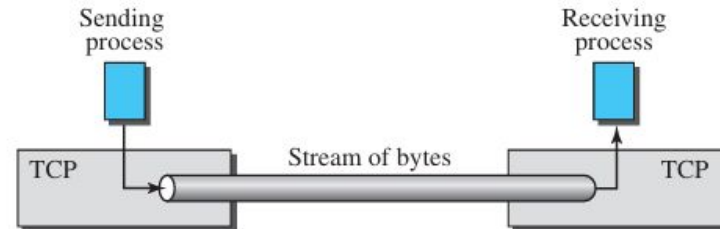
- Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol. TCP explicitly defines connection establishment, data transfer, and connection tear down phases to provide a connection-oriented service.
- TCP uses a combination of GBN(Go Back N Protocol) and SR(Selective Repeat) protocols to provide reliability.
- To achieve this goal, TCP uses checksum (for error detection), retransmission of lost or corrupted packets, cumulative and selective acknowledgments, and timers.
- In this section, we first discuss the services provided by TCP; we then discuss the TCP features in more detail.
- TCP is the most common transport-layer protocol in the Internet.

4.5.1. TCP Services

Process-to-Process Communication

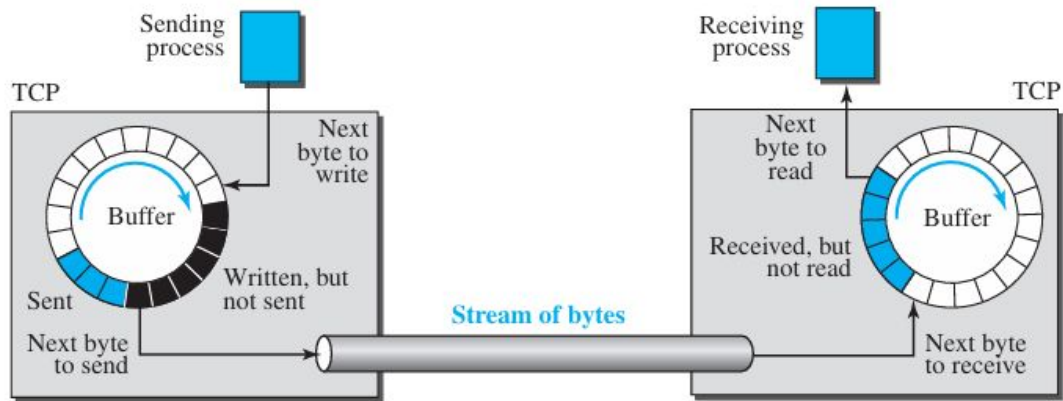
Stream Delivery Service

Figure 24.4 *Stream delivery*



Sending and Receiving Buffers

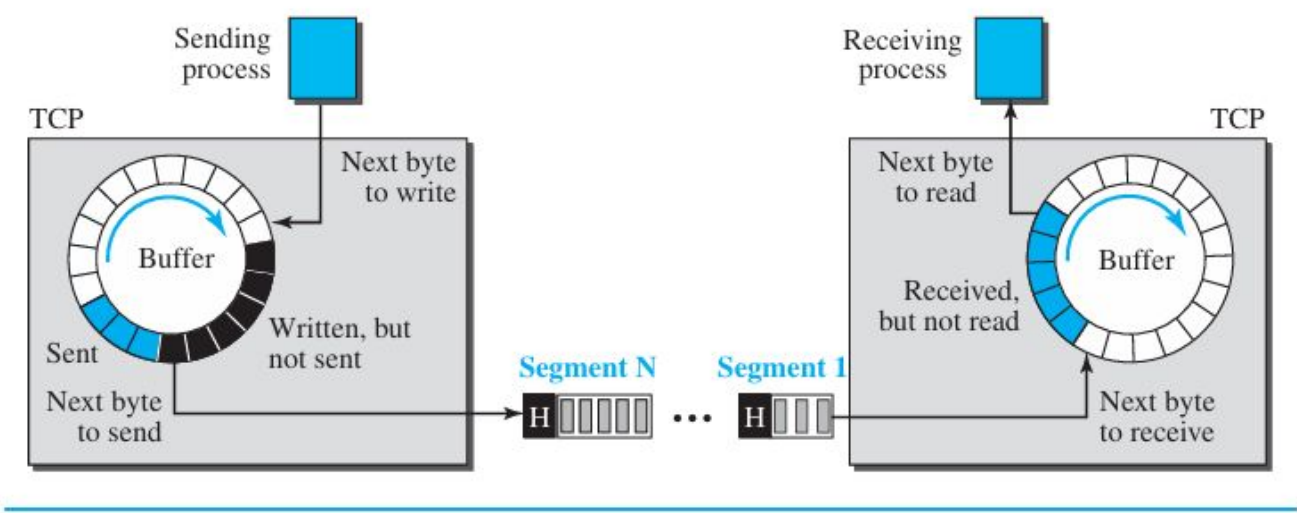
Figure 24.5 *Sending and receiving buffers*



Segments

- At the transport layer, TCP groups a number of bytes together into a packet called a segment.

Figure 24.6 TCP segments



- Full-Duplex Communication
- Multiplexing and Demultiplexing
- Connection-Oriented Service
- Reliable Service

4.5.2. TCP Features

Numbering System

Byte Number

- For example, if the number happens to be 1057 and the total data to be sent is 6000 bytes, the bytes are numbered from 1057 to 7056. We will see that byte numbering is used for flow and error control.
- The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with an arbitrarily generated number

Sequence Number

- Example 24.7** Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Acknowledgment Number

Segment 1	→	Sequence Number:	10001	Range:	10001	to	11000
Segment 2	→	Sequence Number:	11001	Range:	11001	to	12000
Segment 3	→	Sequence Number:	12001	Range:	12001	to	13000
Segment 4	→	Sequence Number:	13001	Range:	13001	to	14000
Segment 5	→	Sequence Number:	14001	Range:	14001	to	15000

4.5.3. Segment

- A packet in TCP is called a segment.

Format

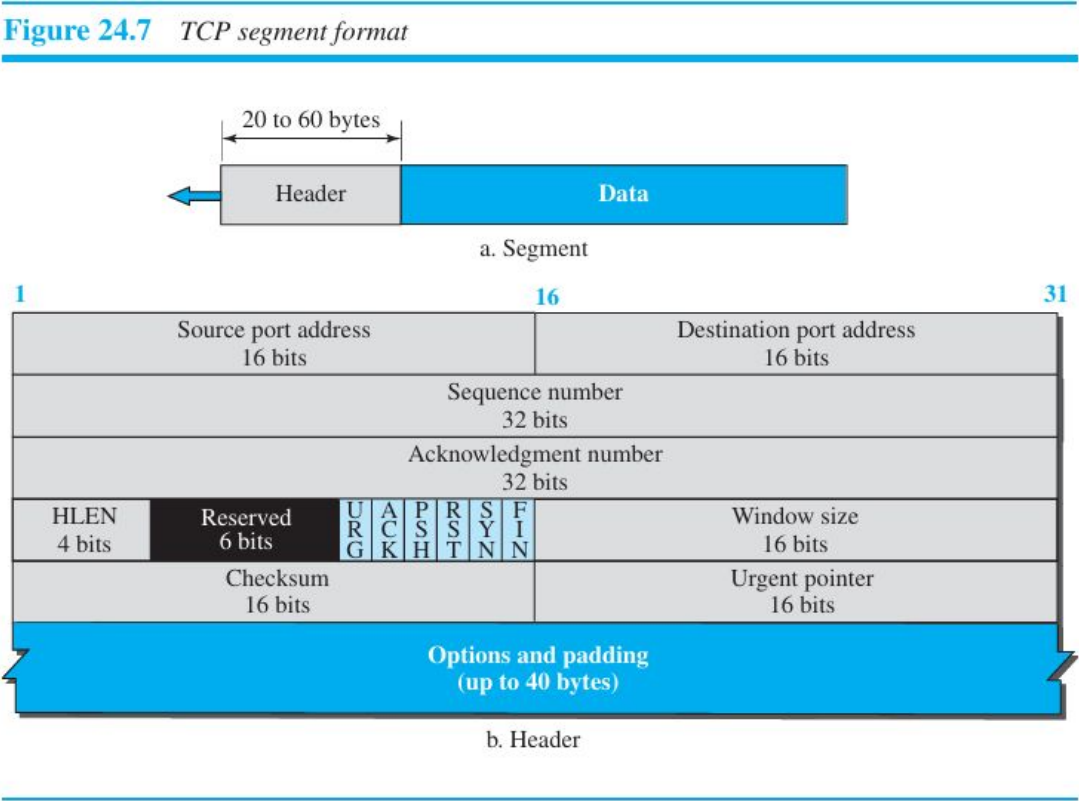


Figure 24.8 *Control field*

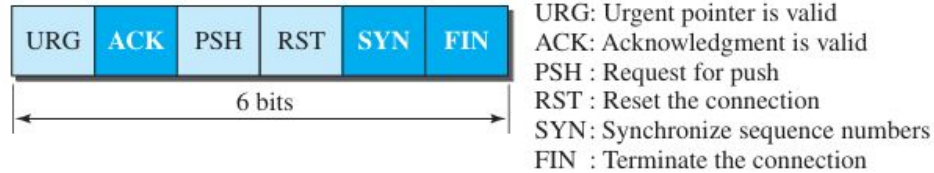
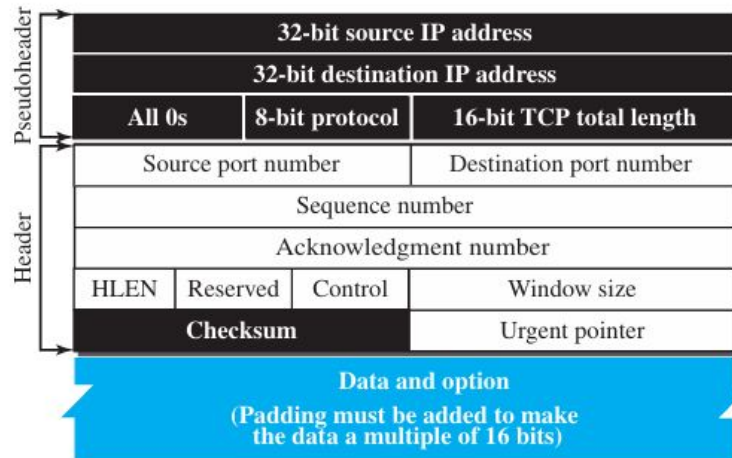


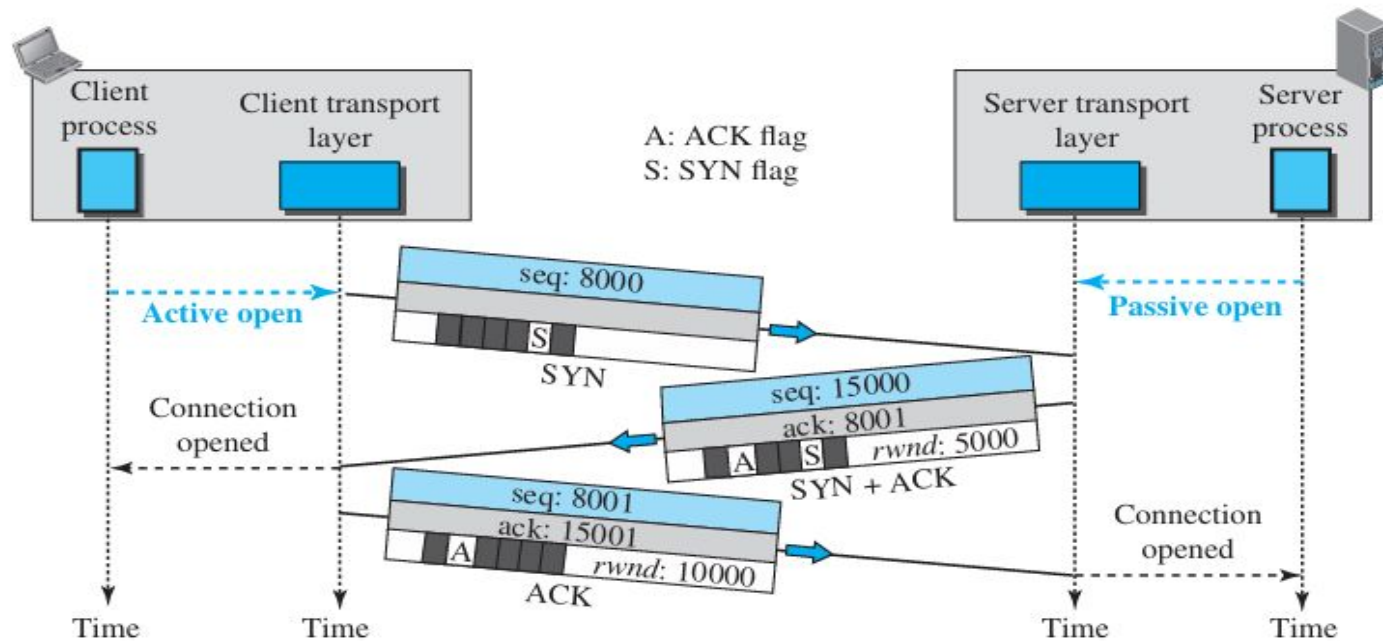
Figure 24.9 *Pseudoheader added to the TCP datagram*



4.5.4. A TCP Connection

- In TCP, connection-oriented transmission requires three phases: **connection establishment, data transfer, and connection termination**
- **Connection Establishment**
 - **Connection Establishment**
 - **passive open.**
 - **active open.**
 - TCP can now start the three-way handshaking process, as shown in Figure 24.10.

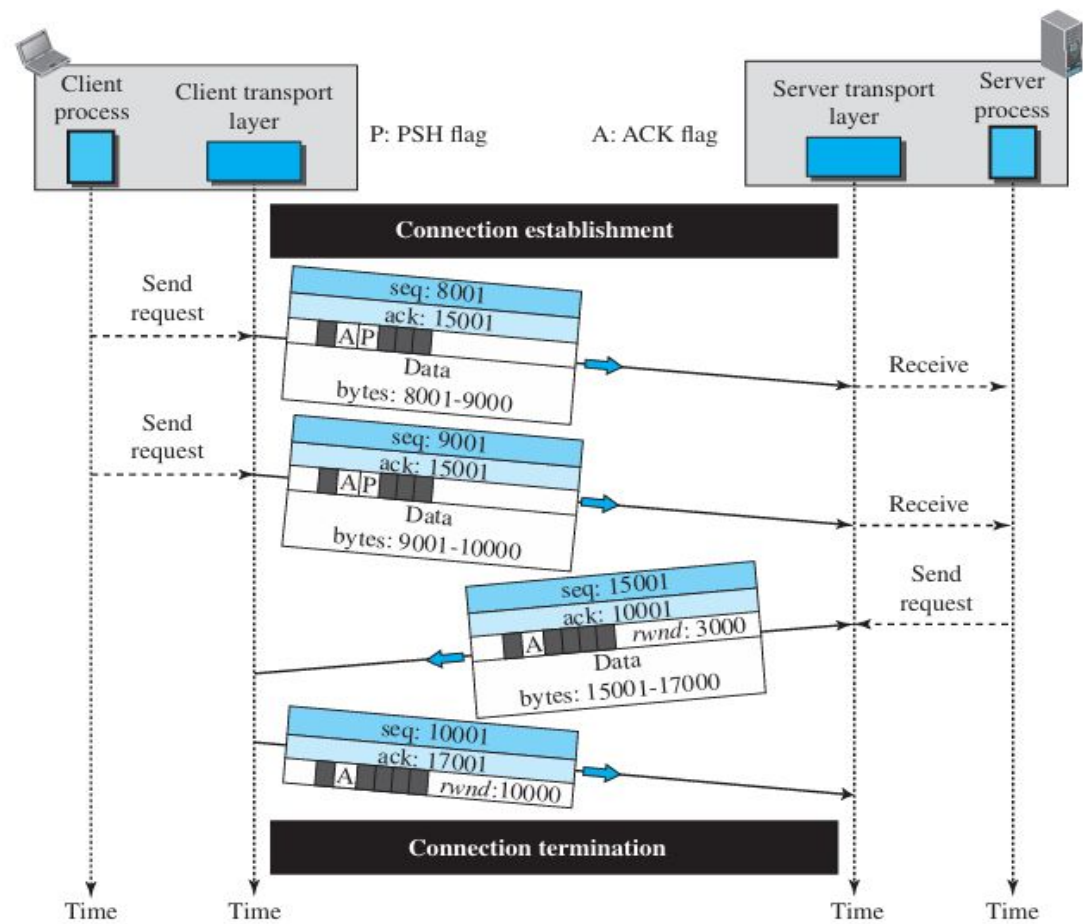
Figure 24.10 *Connection establishment using three-way handshaking*



- 1. The client sends the first segment, a SYN segment, in which only the SYN flag is set.**
 - A SYN segment cannot carry data, but it consumes one sequence number
- 2. The server sends the second segment, a SYN + ACK segment with two flag bits set as: SYN and ACK.**
 - A SYN + ACK segment cannot carry data, but it does consume one sequence number.
- 3. The client sends the third segment.**
 - An ACK segment, if carrying no data, consumes no sequence number.

SYN Flooding Attack

Figure 24.11 Data transfer



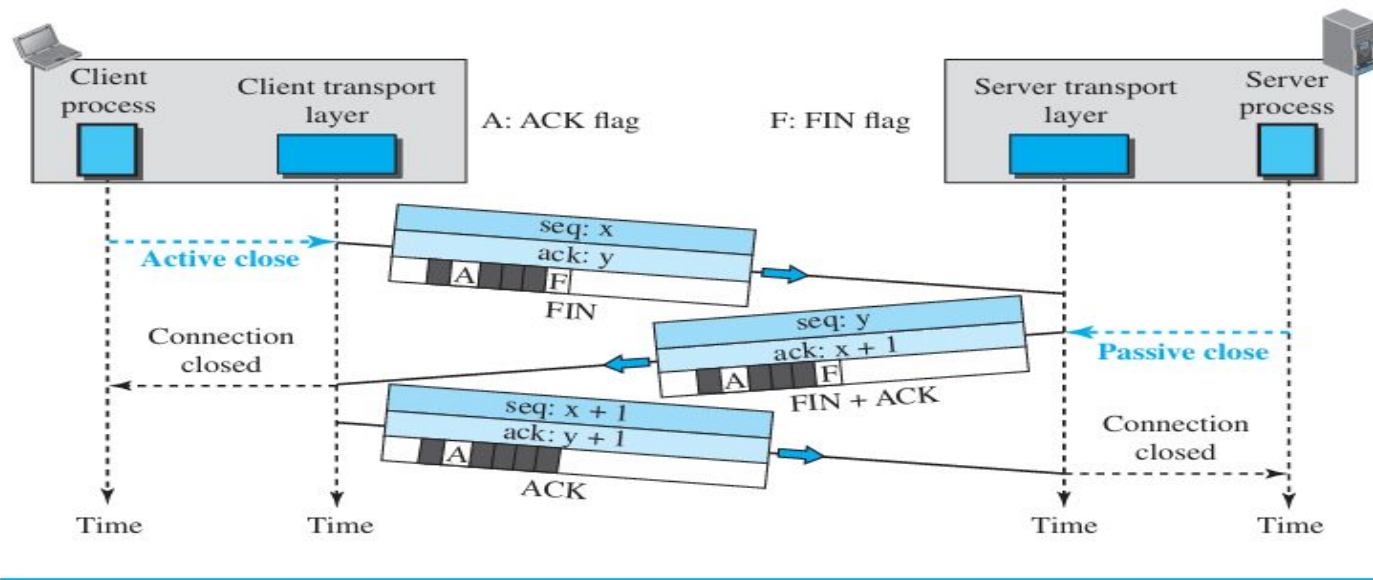
Data Transfer

- Now suppose **both A and B are sending data to each other.**
- **When B receives data from A, instead of sending an immediate ACK in a separate packet, it waits a short time (if possible) until it also has data to send to A.**
- **Then, it attaches (piggybacks) the ACK onto that outgoing data packet.**
- **So, one TCP segment carries both:**
 - **B's data (payload), and**
 - **ACK for A's earlier data.**
- **Pushing Data**
- **Urgent Data**
- **For example**, if the segment sequence number is 15000 and the value of the urgent pointer is 200, the first byte of urgent data is the byte 15000 and the last byte is the byte 15200. The rest of the bytes in the segment (if present) are nonurgent.
-

Connection Termination

Three-Way Handshaking

Figure 24.12 Connection termination using three-way handshaking



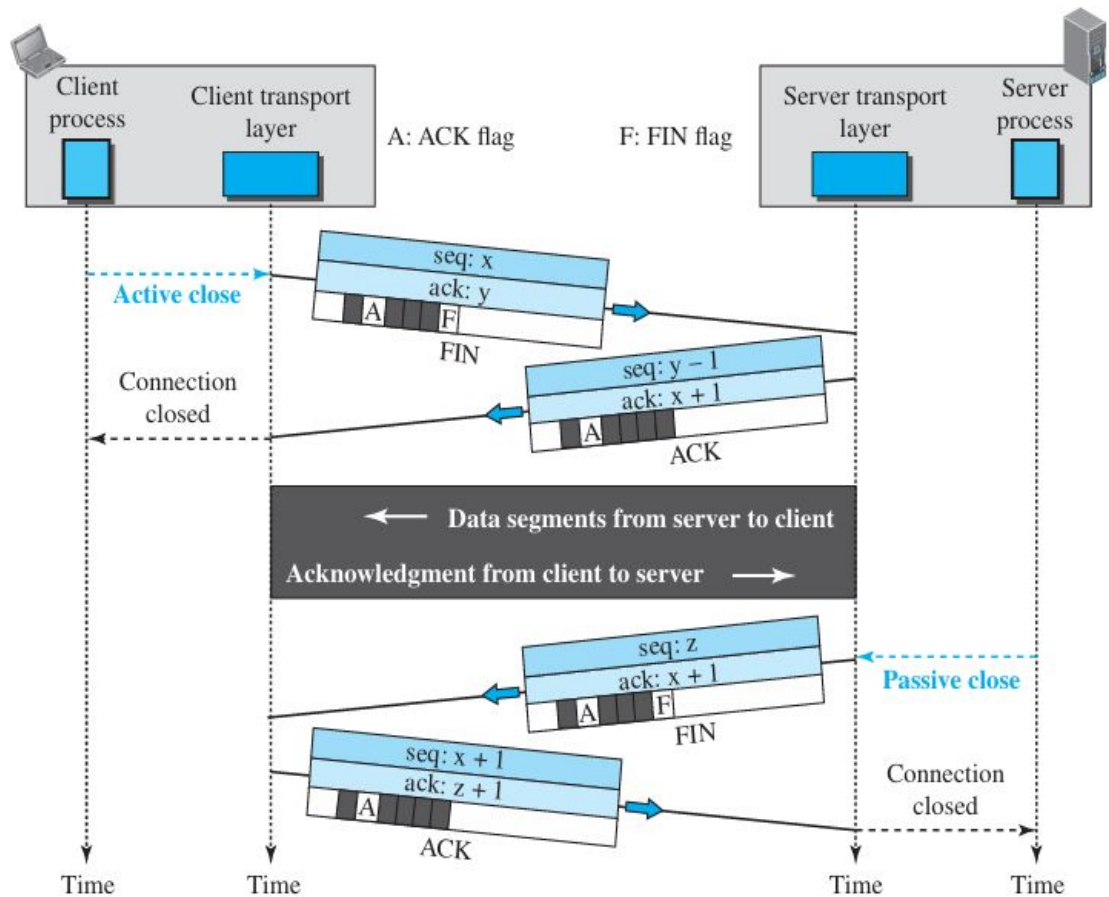
Connection Termination

Half-Close

Connection Reset

- TCP at one end may deny a connection request, may abort an existing connection, or may terminate an idle connection. All of these are done with the RST (reset) flag.

Figure 24.13 Half-close

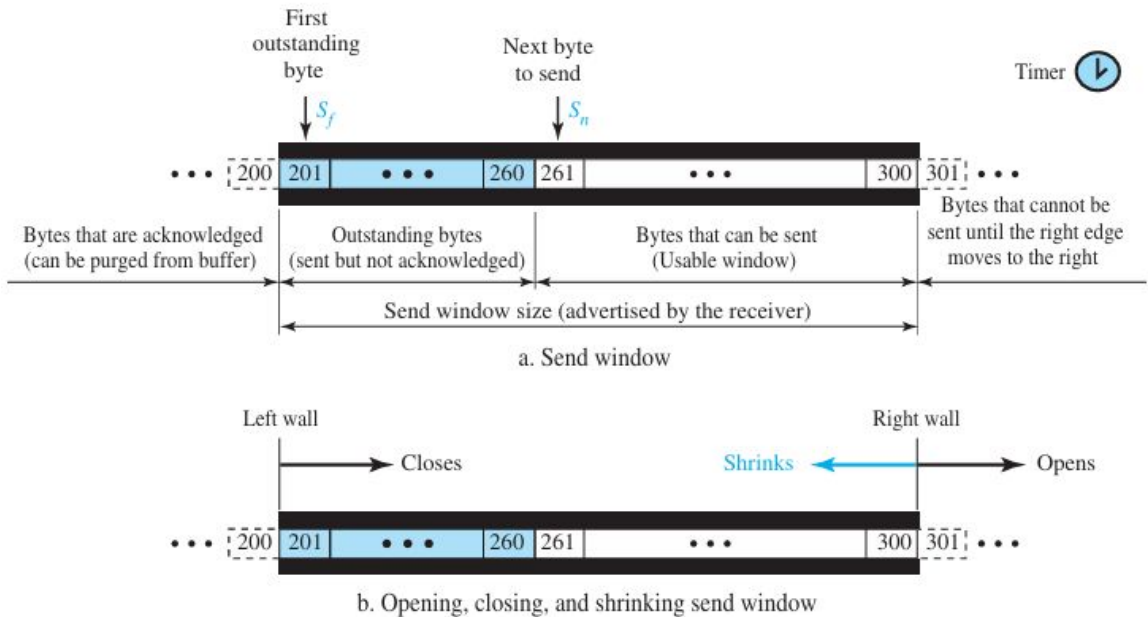


4.5.5. Windows in TCP

- TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication.

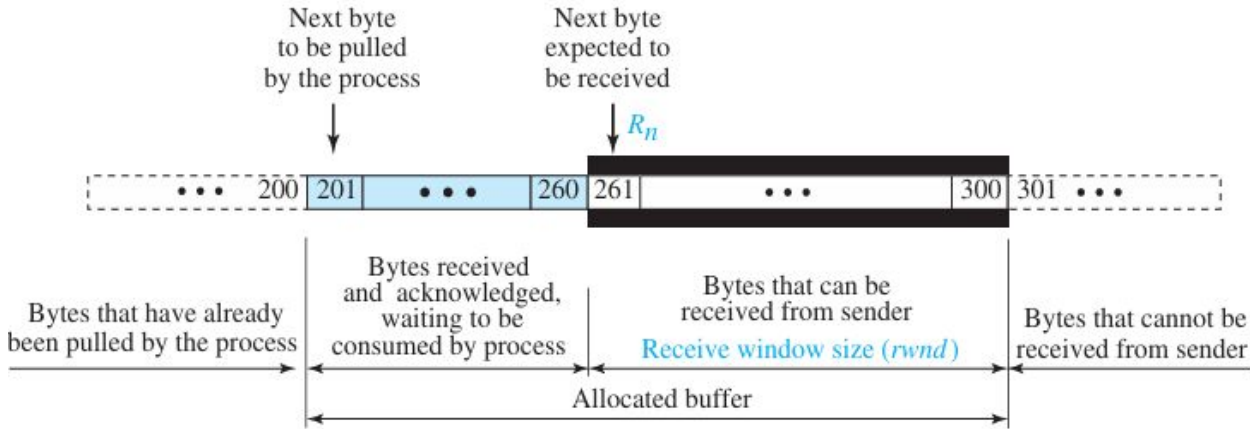
Sending Window

Figure 24.17 *Send window in TCP*

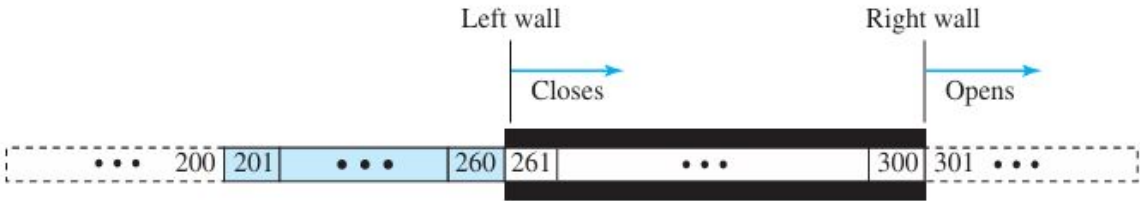


Receiving Window

Figure 24.18 *Receive window in TCP*



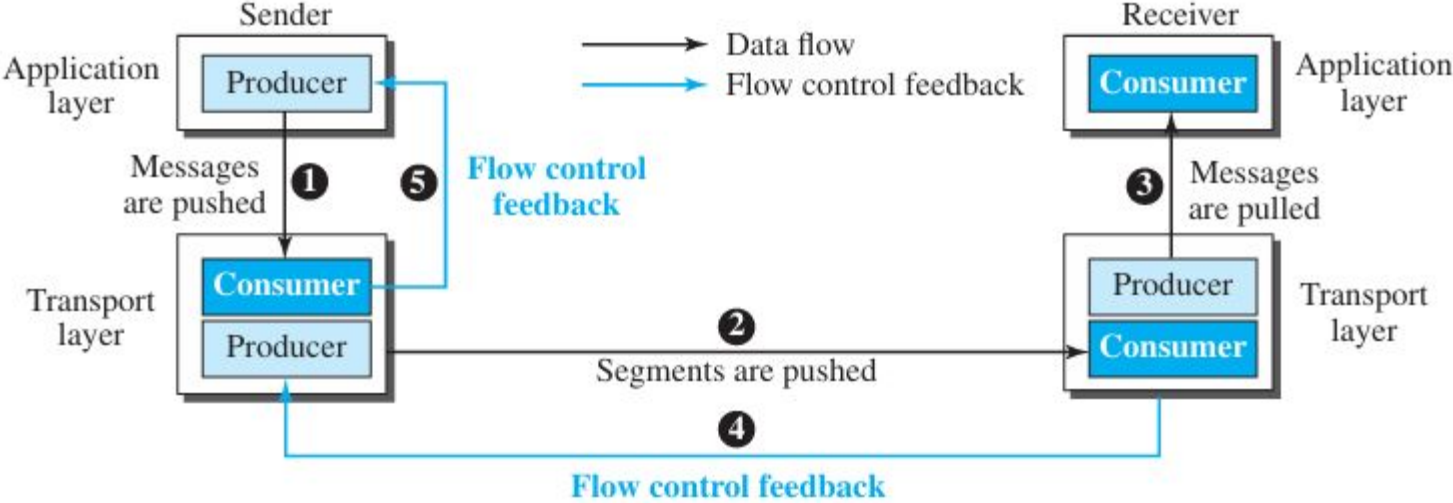
a. Receive window and allocated buffer



b. Opening and closing of receive window

4.5.8. Flow Control

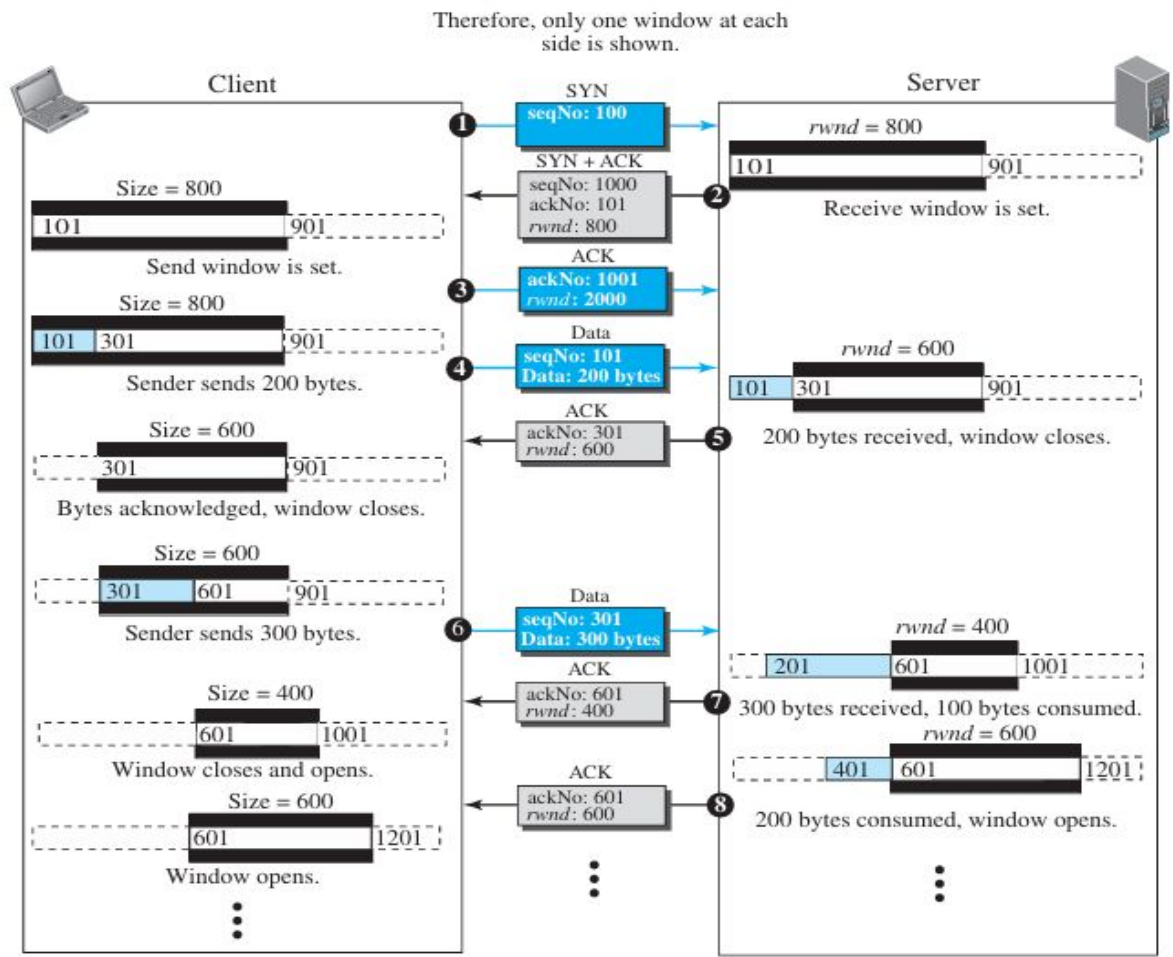
Figure 24.19 *Data flow and flow control feedbacks in TCP*



Opening and Closing Windows

- Eight segments are exchanged between the client and server:

Figure 24.20 An example of flow control



Opening and Closing Windows

Step	Event	rwnd Change	Explanation
1	Client starts (seq=100)	—	Connection setup
2	Server replies (rwnd=800)	800	Server buffer size
3	Client sets send window = 800	—	Ready to send
4	Client sends 200 bytes	800 → 600	200 bytes stored
5	Server ACKs (up to 300)	600	No data read yet
6	Client sends 300 bytes	600 → 400	300 arrived, 100 read
7	Server ACKs again	400	Advertises smaller window
8	Server reads 200 bytes	400 → 600	Window increases again

Shrinking of Windows

- As we said before, the receive window cannot shrink. The send window, on the other hand, can shrink if the receiver defines a value for `rwnd` that results in shrinking the window.
- The receiver needs to keep the following relationship between the last and new acknowledgment and the last and new `rwnd` values to prevent shrinking of the send window.

$$\text{new ackNo} + \text{new rwnd} \geq \text{last ackNo} + \text{last rwnd}$$

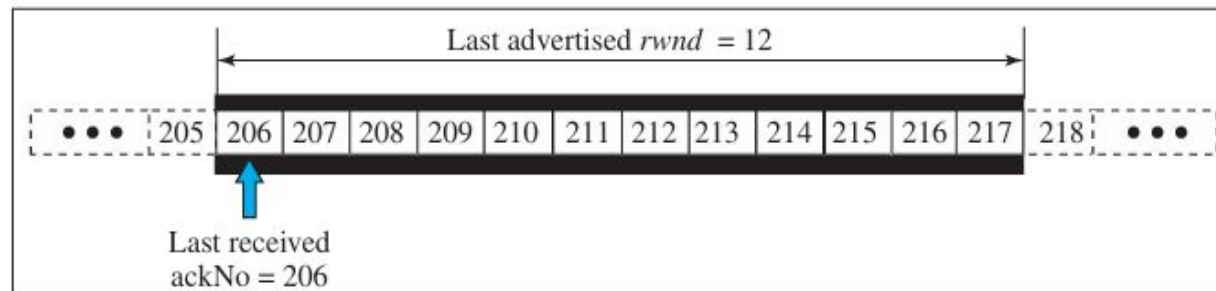
- The left side of the inequality represents the new position of the right wall with respect to the sequence number space; the right side shows the old position of the right wall.
- The relationship shows that the right wall should not move to the left.
- The inequality is a mandate for the receiver to check its advertisement.

Example 24.8

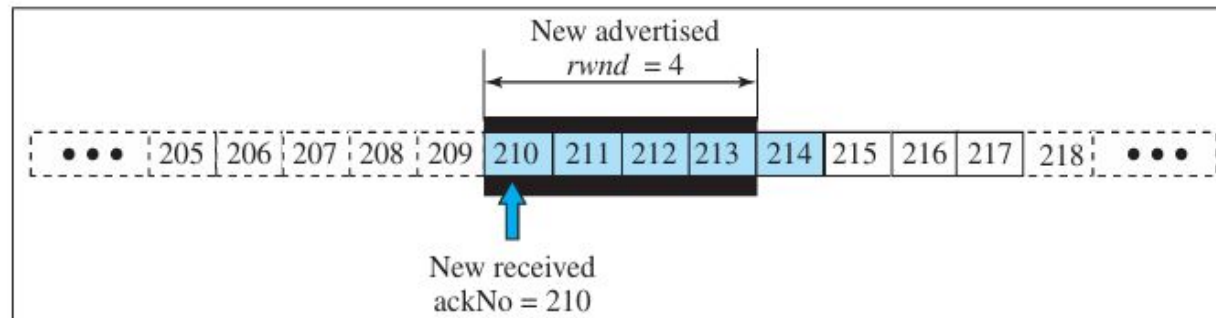
Figure 24.21 shows the reason for this mandate.

Figure 24.21 Example 24.8

- Part a of the figure shows the values of the last acknowledgment and *rwnd*.
- Part b shows the situation in which the sender has sent bytes 206 to 214.



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Window Shutdown

- **Window shutdown** happens when the receiver's buffer becomes completely full, so it advertises a **window size of zero** ($rwnd = 0$) to the sender.
- This tells the sender to **stop sending more data temporarily**.
- The sender must wait until the receiver advertises a **nonzero window** again before resuming transmission.

Silly Window Syndrome

- **Silly Window Syndrome (SWS)** happens when data is sent or received in **very small chunks**, making the network extremely inefficient.
- If the **sender** produces data slowly (like 1 byte at a time), it sends tiny segments. Or if the **receiver** processes data slowly, it keeps advertising a **very small window** (like 1 byte free), forcing the sender to send tiny packets.
- This causes **waste of bandwidth**, because each small data packet carries a large TCP/IP header (40 bytes) — meaning more overhead than useful data.
- SWS means the network is busy sending mostly headers, not actual data — making communication very inefficient.

Syndrome Created by the Sender

Syndrome Created by the Receiver

- **Solutions for Receiver-Side Silly Window Syndrome:**

1. **Clark's Solution:**

- Acknowledge incoming data immediately but **advertise window = 0** until there's enough free space (either half the buffer or enough for a full segment)

2. **Delayed Acknowledgment:**

- **Wait before sending ACK** until there's enough buffer space, preventing the sender from sending more small segments too early.

3. **Benefits:**

- Reduces unnecessary small segments and **cuts down network traffic**.

4. **Drawback & Balance:**

- Delay can cause **unnecessary retransmissions**, so TCP limits the delay — **ACKs must be sent within 500 ms**.

4.5.9. Error control